

Use Case 5: Clickstream Real-Time Analytics (Stream Processing)

Snowflake + PySpark Structured Streaming + AWS Glue + Kafka

1. Overview

WebMart, a large e-commerce retailer, captures millions of clickstream events every day as users browse products, search the catalog, add items to their carts, and complete purchases. The product analytics team needs **real-time** visibility into user behavior to:

- Detect trending products within minutes, not hours
- Identify abandoned carts and trigger re-engagement workflows
- Measure page performance and bounce rates continuously
- Understand the conversion funnel in near real-time

In this use case you will build a **streaming pipeline** that ingests raw click events from Apache Kafka, sessionizes them using PySpark Structured Streaming, computes real-time analytics, and sinks the results into Snowflake for dashboarding and ad-hoc analysis.

2. Business Scenario

WebMart's web and mobile applications emit a JSON event for every user interaction: page views, clicks, searches, add-to-cart actions, and purchases. These events are published to a **Kafka topic** called `clickstream`.

Your mission:

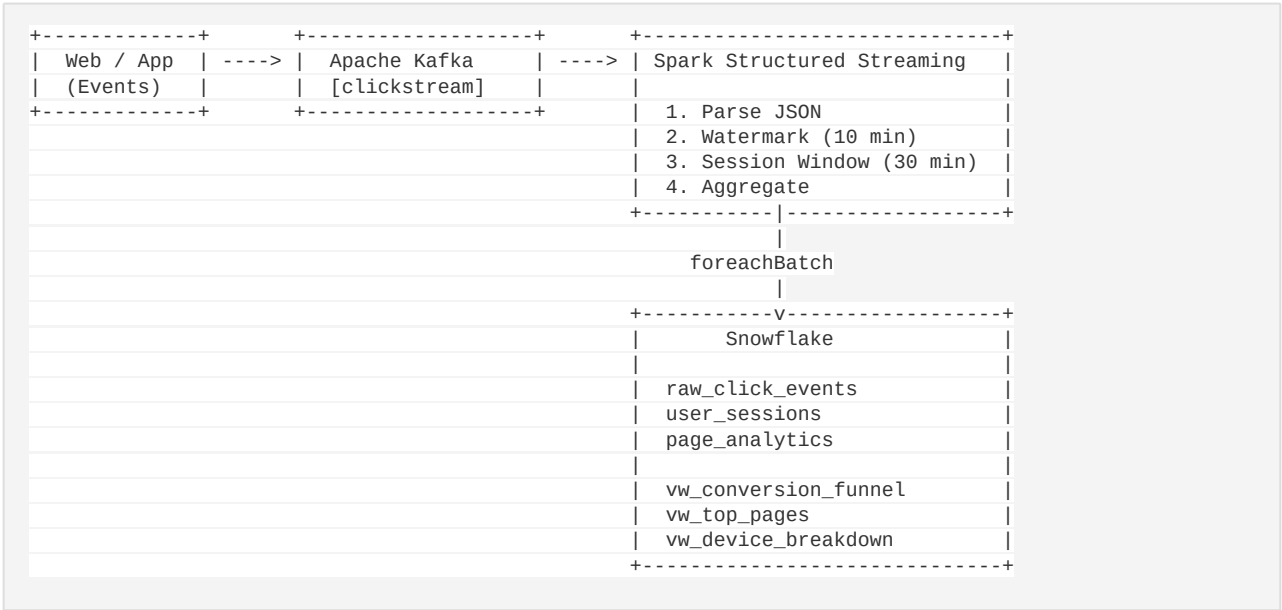
1. Consume the raw event stream from Kafka.
 2. Parse and validate the JSON payload.
 3. **Sessionize** the events: group events that belong to the same browsing session using a **30-minute inactivity gap**.
 4. Handle **late-arriving data** gracefully using watermarks.
 5. Compute per-session metrics (duration, page count, whether a purchase occurred).
 6. Compute page-level analytics (views per URL, unique visitors, bounce rate).
 7. Write all results to Snowflake via `foreachBatch`.
-

3. Learning Objectives

After completing this use case you will be able to:

#	Objective
1	Consume and deserialize JSON messages from a Kafka topic using Spark Structured Streaming
2	Apply watermarking to tolerate late-arriving events
3	Use session windows (<code>session_window</code>) to group events into browsing sessions
4	Perform streaming aggregations (counts, distinct counts, conditional flags)
5	Implement <code>foreachBatch</code> to write streaming results to Snowflake
6	Design Snowflake tables and views for real-time clickstream analytics
7	Reason about event time vs. processing time and out-of-order data

4. Architecture Diagram



Data Flow Summary:

1. The Kafka producer (provided) simulates realistic user browsing sessions and publishes JSON events to the `clickstream` topic.
2. Spark Structured Streaming reads from Kafka, parses the JSON, applies a watermark, and performs two streaming aggregations:
3. **Session-level:** groups events into sessions using `session_window` with a 30-minute gap, then computes session metrics.
4. **Page-level:** groups by `page_url` in tumbling windows to compute page view counts and unique visitor counts.

- 5. A `foreachBatch` sink writes micro-batch results to Snowflake tables.
- 6. Snowflake views provide business-ready analytics on top of the raw tables.

5. Dataset Description

Source: `clickstream` Kafka topic (JSON)

Each message is a single JSON object with the following schema:

Field	Type	Description	Example
<code>event_id</code>	string	Unique event identifier	EVT_00142
<code>session_id</code>	string	Client-assigned session ID	SESS_1023
<code>user_id</code>	string (nullable)	Authenticated user ID; null for anonymous	USR_205
<code>page_url</code>	string	Page path or URL	/products/laptop-pro-15
<code>action</code>	string	Type of interaction	page_view, click, add_to_cart, remove_from_cart, search, purchase
<code>referrer</code>	string (nullable)	Previous page or external referrer	https://google.com
<code>device_type</code>	string	Device category	desktop, mobile, tablet
<code>browser</code>	string	Browser name	Chrome, Firefox, Safari, Edge
<code>ip_address</code>	string	Client IP address	192.168.1.45
<code>timestamp</code>	string (ISO 8601)	Event timestamp in UTC	2024-08-10T14:22:35Z

What is a "session"?

A **session** is a sequence of events from the same `session_id` where no two consecutive events are more than **30 minutes** apart. If a user is inactive for more than 30 minutes and then returns, that constitutes a new session.

Sample Data

A sample file with 20 records is provided at `datasets/use_case_5/sample_clickstream.json` for local testing before connecting to Kafka.

6. Tasks

Task 1: Start Kafka and Run the Clickstream Producer

Objective: Get the data flowing.

1. Ensure your Kafka cluster is running (e.g., via Docker Compose or a managed service).
 2. Create the `clickstream` topic with at least 3 partitions: `bash kafka-topics.sh --create --topic clickstream \ --bootstrap-server localhost:9092 \ --partitions 3 --replication-factor 1`
 3. Run the provided producer: `bash python streaming_utils/kafka_producer_clickstream.py \ --bootstrap-servers localhost:9092 \ --topic clickstream \ --num-users 20 \ --events-per-second 5`
 4. Verify events are flowing: `bash kafka-console-consumer.sh --bootstrap-server localhost:9092 \ --topic clickstream --from-beginning --max-messages 5`
-

Task 2: Set Up Snowflake Tables

Objective: Create the target tables and views in Snowflake.

Create the following objects in your Snowflake database:

- `raw_click_events` -- stores every raw event
- `user_sessions` -- one row per session with computed metrics
- `page_analytics` -- aggregated page-level statistics
- Views: `vw_conversion_funnel`, `vw_top_pages`, `vw_device_breakdown`

Refer to `solutions/use_case_5/snowflake_setup.sql` after you have attempted your own DDL.

Key design decisions to think about: - What should the clustering key be for `raw_click_events`? - Should `user_sessions.actions_json` be stored as `VARIANT` or `VARCHAR`? - How will you handle upserts for sessions that span multiple micro-batches?

Task 3: Read from Kafka Clickstream Topic

Objective: Create a Spark Structured Streaming `DataFrame` that reads from Kafka.

```
raw_stream = (  
    spark.readStream
```

```
.format("kafka")
.option("kafka.bootstrap.servers", "localhost:9092")
.option("subscribe", "clickstream")
.option("startingOffsets", "latest")
.option("failOnDataLoss", "false")
.load()
)
```

The resulting DataFrame has Kafka metadata columns (key, value, topic, partition, offset, timestamp, timestampType). The actual event payload is in the `value` column as binary.

Your task: Cast `value` to a string and verify you can see the JSON payloads.

Task 4: Parse JSON and Extract Fields

Objective: Deserialize the JSON string into a structured DataFrame.

1. Define a `StructType` schema matching the JSON fields.
2. Use `from_json()` to parse the `value` column.
3. Select and flatten all fields.
4. Cast the `timestamp` string to a proper `TimestampType`.

```
from pyspark.sql.types import StructType, StructField, StringType, TimestampType
from pyspark.sql.functions import from_json, col

click_schema = StructType([
    StructField("event_id", StringType()),
    StructField("session_id", StringType()),
    StructField("user_id", StringType()),
    StructField("page_url", StringType()),
    StructField("action", StringType()),
    StructField("referrer", StringType()),
    StructField("device_type", StringType()),
    StructField("browser", StringType()),
    StructField("ip_address", StringType()),
    StructField("timestamp", StringType()),
])
```

Your task: Complete the parsing pipeline and output a clean DataFrame with properly typed columns.

Task 5: Implement Watermarking for Late Data

Objective: Add a watermark to tolerate events that arrive up to 10 minutes late.

In real-world clickstream pipelines, events can arrive out of order due to network delays, mobile app background syncing, or client-side buffering. A **watermark** tells Spark how long to wait for late data before finalizing aggregation results.

```
events_with_watermark = parsed_events.withWatermark("event_timestamp", "10 minutes")
```

Key concepts: - The watermark is applied on the **event time** column (from the JSON payload), not the Kafka ingestion timestamp. - Events arriving more than 10 minutes after the watermark has advanced will be dropped. - This is a trade-off: larger watermarks provide more completeness but increase latency and state size.

Your task: Apply the watermark and explain in comments why you chose the event timestamp over the Kafka timestamp.

Task 6: Sessionize Events Using Session Windows

Objective: Group events into sessions using `session_window` with a 30-minute gap.

Spark's `session_window` function creates dynamic windows: a new window starts with the first event and extends as long as events keep arriving within the gap duration. If no event arrives for 30 minutes, the window closes.

```
from pyspark.sql.functions import session_window

session_agg = (
    events_with_watermark
    .groupBy(
        session_window("event_timestamp", "30 minutes"),
        "session_id"
    )
    .agg(...)
)
```

Your task: Complete the aggregation to compute: - Session start time and end time (from the window) - Session duration in seconds - Total number of events (`page_count`) - Number of distinct pages visited - Count of each action type - Whether the session included a purchase action (boolean converted) - The device type and `user_id` (use `first()`)

Task 7: Compute Per-Session Metrics

Objective: Enrich the session aggregation with derived business metrics.

From the session-level DataFrame produced in Task 6, compute:

Metric	Formula
duration_seconds	<code>unix_timestamp(session_end) - unix_timestamp(session_start)</code>
is_bounce	<code>page_count == 1</code>
converted	Session contains at least one purchase action
actions_json	JSON string summarizing action counts, e.g., <code>{"page_view": 3, "add_to_cart": 1}</code>

Your task: Add these computed columns to the session DataFrame.

Task 8: Compute Page-Level Analytics

Objective: Create a second streaming aggregation for page-level metrics.

Group by `page_url` in a **tumbling window** (e.g., 5 minutes) with the same watermark:

Metric	Description
view_count	Count of <code>page_view</code> actions
unique_visitors	Approximate distinct count of <code>user_id</code> (or <code>ip_address</code> for anonymous)
avg_time_on_page	Average time between arriving on the page and the next event

<code>bounce_rate</code>	Fraction of sessions where the page was the only page viewed
--------------------------	--

Your task: Build this aggregation. Note that some metrics (like bounce rate and time on page) may require additional joins or stateful processing. Start with `view_count` and `unique_visitors`, then attempt the others.

Task 9: Write Results to Snowflake via foreachBatch

Objective: Sink all streaming results to Snowflake.

Use `foreachBatch` to write each micro-batch to Snowflake using the Snowflake Spark connector:

```
def write_to_snowflake(df, epoch_id, table_name):
    (
        df.write
        .format("snowflake")
        .options(**snowflake_options)
        .option("dbtable", table_name)
        .mode("append")
        .save()
    )

# Example for raw events
raw_query = (
    parsed_events.writeStream
    .foreachBatch(lambda df, eid: write_to_snowflake(df, eid, "RAW_CLICK_EVENTS"))
    .option("checkpointLocation", "/tmp/checkpoints/raw_events")
    .outputMode("append")
    .start()
)
```

Your task: 1. Set up Snowflake connection options (account, user, password, database, schema, warehouse). 2. Write three streaming queries: one for raw events, one for sessions, one for page analytics. 3. Ensure each query has a unique checkpoint location. 4. Consider: what output mode should each query use? (append vs. update vs. complete)

Task 10: Handle Late Data and Verify Completeness

Objective: Validate that late-arriving events are handled correctly.

1. The Kafka producer generates ~5% of events with timestamps in the past (simulating late arrival).
2. Verify that events within the 10-minute watermark tolerance are included in session aggregations.
3. Check Snowflake for:
 4. Sessions that were updated when late data arrived
 5. Any gaps in the event sequence
 6. Experiment: change the watermark to 1 minute and observe which late events are dropped.

Your task: - Run the pipeline for at least 10 minutes. - Query Snowflake to confirm data completeness. - Document your observations about late data handling.

7. Expected Outputs

After completing all tasks, you should have:

Snowflake Tables

`raw_click_events` -- every event, one row per click:

EVENT_ID	SESSION_ID	USER_ID	PAGE_URL	ACTION	TIMESTAMP
EVT_00001	SESS_1001	USR_201	/home	page_view	2024-08-10 14:22:35.000
EVT_00002	SESS_1001	USR_201	/search?q=laptop	search	2024-08-10 14:23:10.000
EVT_00003	SESS_1001	USR_201	/products/laptop-pro-15	page_view	2024-08-10 14:24:02.000

`user_sessions` -- one row per session:

SESSION_ID	USER_ID	SESSION_START	SESSION_END	DURATION_S	PAGE_COUNT	CONVERTED
SESS_1001	USR_201	2024-08-10 14:22:35.000	2024-08-10 14:30:12.000	457	6	TRUE
SESS_1002	USR_203	2024-08-10 14:23:55.000	2024-08-10 14:26:15.000	140	3	FALSE

`page_analytics` -- aggregated page metrics:

PAGE_URL	VIEW_COUNT	UNIQUE_VISITORS	AVG_TIME_ON_PAGE	BOUNCE_RATE
/home	1245	892	35.2	0.18
/products/laptop-pro-15	567	423	62.8	0.05
/cart	234	198	45.1	0.12

Streaming Queries

Three active streaming queries writing to Snowflake: 1. Raw events (append mode) 2. Session aggregations (update mode) 3. Page analytics (update mode)

8. Hints

Level 1 Hints (Gentle Nudges)

- When parsing JSON from Kafka, remember that the value column is binary. Cast it to `StringType` first.
- For session windows, the gap duration is specified as a string like `"30 minutes"`.
- The `converted` flag can be computed using `count(when(col("action") == "purchase", True))` inside your aggregation.
- Use `approx_count_distinct` for `unique_visitors` -- it is faster than exact `countDistinct` on large streams.

Level 2 Hints (More Specific)

- Your full parsing pipeline should look like: `python parsed = ( raw_stream .selectExpr("CAST(value AS STRING) as json_str") .select(from_json("json_str", click_schema).alias("data")) .select("data.*") .withColumn("event_timestamp", to_timestamp("timestamp"))) )`
- For session aggregation output mode, use `"update"` -- not `"complete"` (complete mode requires all data in memory) and not `"append"` (append waits until the watermark passes the window end).
- The `actions_json` column can be built with `to_json(struct(...))` where you count each action type.
- When writing sessions to Snowflake, consider using `MERGE` instead of `APPEND` to handle sessions that span multiple micro-batches. In `foreachBatch`, you can run a SQL merge via the Snowflake connector.

Level 3 Hints (Nearly the Answer)

1. Complete session aggregation:

```
python session_metrics = ( events_with_watermark .groupBy(
    session_window("event_timestamp", "30 minutes"), "session_id" ) .agg(
    first("user_id").alias("user_id"), first("device_type").alias("device_type"),
    count("*").alias("page_count"), countDistinct("page_url").alias("distinct_pages"),
    sum(when(col("action") == "page_view", 1).otherwise(0)).alias("views"), sum(when(col("action") ==
    "add_to_cart", 1).otherwise(0)).alias("add_to_carts"), sum(when(col("action") == "purchase",
    1).otherwise(0)).alias("purchases"), (max("event_timestamp").cast("long") -
    min("event_timestamp").cast("long")).alias("duration_seconds"), ) .withColumn("converted",
    col("purchases") > 0) .withColumn("session_start", col("session_window.start"))
    .withColumn("session_end", col("session_window.end"))) )
```
2. For the Snowflake foreachBatch writer, drop the session_window struct column before writing:

```
python df_to_write
= df.drop("session_window")
```

9. Validation Queries

Run these queries in Snowflake after the pipeline has been running for at least 5 minutes.

Check raw event counts

```
SELECT COUNT(*) AS total_events,
       COUNT(DISTINCT session_id) AS unique_sessions,
       COUNT(DISTINCT user_id) AS unique_users,
       MIN(event_timestamp) AS earliest_event,
       MAX(event_timestamp) AS latest_event
FROM raw_click_events;
```

Verify session metrics

```
SELECT session_id,
       user_id,
       session_start,
       session_end,
       duration_seconds,
       page_count,
       converted
FROM user_sessions
ORDER BY session_start DESC
LIMIT 20;
```

Check conversion funnel

```
SELECT * FROM vw_conversion_funnel;
```

Find abandoned carts

```
SELECT session_id, user_id, page_count, duration_seconds
FROM user_sessions
WHERE converted = FALSE
  AND session_id IN (
    SELECT DISTINCT session_id
    FROM raw_click_events
    WHERE action = 'add_to_cart'
)
```

```
ORDER BY duration_seconds DESC;
```

Top pages by views

```
SELECT * FROM vw_top_pages LIMIT 10;
```

Device breakdown

```
SELECT * FROM vw_device_breakdown;
```

Verify late-data handling

```
-- Find events that arrived late (event timestamp significantly before insert timestamp)
SELECT event_id, session_id, event_timestamp, inserted_at,
       DATEDIFF('minute', event_timestamp, inserted_at) AS delay_minutes
FROM raw_click_events
WHERE DATEDIFF('minute', event_timestamp, inserted_at) > 1
ORDER BY delay_minutes DESC
LIMIT 20;
```

10. Additional Challenges (Optional)

If you finish early, try these extensions:

1. **Real-time alerting:** Trigger an alert when a product page exceeds 100 views in a 5-minute window.
2. **A/B test analysis:** Add a `variant` field to the clickstream and compute conversion rates per variant in real time.
3. **Sessionization without `session_window`:** Implement custom sessionization using `flatMapGroupsWithState` for more control.
4. **Exactly-once semantics:** Configure Kafka and Spark for exactly-once delivery and verify no duplicates in Snowflake.
5. **Backpressure handling:** Configure `maxOffsetsPerTrigger` and observe how it affects throughput and latency.

References

- [Spark Structured Streaming Programming Guide](#)
- [Session Windows in Spark](#)
- [Snowflake Spark Connector](#)
- [Kafka Documentation](#)
- [Watermarking in Structured Streaming](#)